Ex.no:8	MINI PROJECT – TOURISM & ECONOMIC ANALYTICS DATA ANALYTICS

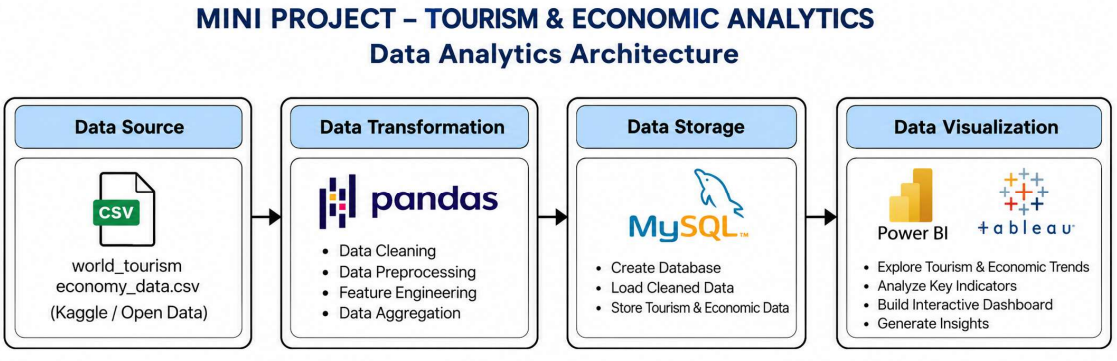
Aim:

To clean, transform, store, and analyze global tourism and economic data using Python, MySQL, Power BI and Tableau to derive meaningful insights into tourism arrivals, tourism receipts, tourism expenditure, economic indicators, and country-wise tourism performance.

Algorithm:

1. Load the global tourism and economic dataset from the CSV file using Pandas.
2. Inspect the dataset for missing values and duplicate records.
3. Handle missing categorical values and numerical values using suitable imputation methods.
4. Remove duplicate records and standardize the column names.
5. Create derived attributes such as tourism_balance, arrivals_per_gdp, and period.
6. Store the cleaned and transformed dataset in a MySQL database named tourism_analytics.
7. Connect the MySQL database to Power BI for analysis and reporting.
8. Create DAX measures for total countries, tourist arrivals, tourism receipts, tourism expenditure, average GDP, average inflation, and tourism balance.
9. Develop interactive Power BI and Tableau visualizations to analyze tourism and economic trends.
10. Generate meaningful insights from the dashboards.
11. End the process.

Architecture:



Program:

1. Python – Data Cleaning & Transformation

```
import pandas as pd
from sqlalchemy import create_engine, text
# Load dataset
df = pd.read_csv("fifa_world_cup_2026_player_performance.csv")
# Create a copy for processing
fifa = df.copy()
# Handle missing values
fifa["player_name"] = fifa["player_name"].fillna("Unknown")
fifa["nationality"] = fifa["nationality"].fillna("Unknown")
fifa["team"] = fifa["team"].fillna("Unknown")
fifa["position"] = fifa["position"].fillna("Unknown")
# Remove duplicate records
fifa.drop_duplicates(inplace=True)
# Convert match date
fifa["match_date"] = pd.to_datetime(fifa["match_date"])
# Create derived columns
fifa["goal_contribution"] = (
    fifa["goals"] + fifa["assists"]
)
fifa["pass_efficiency"] = (
    fifa["successful_passes"] /
    fifa["total_passes"]
)
# Standardize column names
fifa.columns = (
    fifa.columns
    .str.lower()
    .str.strip()
    .str.replace(" ", "_")
)
print("Original Shape:", df.shape)
print("Cleaned Shape:", fifa.shape)
fifa.head()
```

Python – Output:

Original dataset shape: 6650 rows × 11 columns.

Cleaned dataset shape: 6650 rows × 14 columns.

The cleaned dataset contains the original attributes together with tourism_balance, arrivals_per_gdp and period

```
Original Shape: (6650, 11)

Missing Values Before Cleaning:
country                    0
country_code               0
year                       0
tourism_receipts        2361
tourism_arrivals        1701
tourism_exports         2536
tourism_departures      4061
tourism_expenditures    2477
gdp                      226
inflation                982
unemployment            2992
dtype: int64

Duplicate Records Before Cleaning: 0

Missing Values After Cleaning:
country                   0
country_code              0
year                      0
tourism_receipts          0
tourism_arrivals          0
tourism_exports           0
tourism_departures        0
tourism_expenditures      0
gdp                       0
inflation                 0
unemployment              0
tourism_balance           0
arrivals_per_gdp          0
period                    0
dtype: int64
```

```
Duplicate Records After Cleaning: 0

Final Shape: (6650, 14)

Data Types After Cleaning:
country                    str
country_code               str
year                     int64
tourism_receipts       float64
tourism_arrivals       float64
tourism_exports        float64
tourism_departures     float64
tourism_expenditures   float64
gdp                    float64
inflation              float64
unemployment           float64
tourism_balance        float64
arrivals_per_gdp       float64
period                category
dtype: object
```

First 5 Cleaned Records:

	country	country_code	year	tourism_receipts	tourism_arrivals	tourism_exports	tourism_departures	tourism_expenditures	gdp	inflation	unemploy
0	Aruba	ABW	1999	7.820000e+08	9.720000e+05	62.542949	4634000.0	9.495387	1.722905e+09	2.280372	
1	Africa Eastern and Southern	AFE	1999	8.034209e+09	1.530938e+07	12.204030	4634000.0	7.760536	2.654293e+11	7.819865	
2	Afghanistan	AFG	1999	1.553000e+09	2.508000e+06	8.306797	4634000.0	5.754790	3.681803e+10	3.629433	
3	Africa Western and Central	AFW	1999	1.443613e+09	3.897975e+06	3.974476	4634000.0	6.147291	1.394683e+11	0.372266	
4	Angola	AGO	1999	3.100000e+07	4.500000e+04	0.583858	4634000.0	2.489638	6.152923e+09	248.195902	

2. MySQL – Database and Data Loading

MySQL:

```
CREATE DATABASE IF NOT EXISTS tourism_analytics;
USE tourism_analytics;
```

Python/Jupyter:

```
from sqlalchemy import create_engine

username = "root"
password = "Harini*2006"
host = "localhost"
port = 3306

# Connect to MySQL database
db_engine = create_engine(
    f"mysql+pymysql://{username}:{password}"
    f"@{host}:{port}/tourism_analytics"
)

# Load cleaned data into MySQL
tourism.to_sql(
    "tourism_data",
    con=db_engine,
    if_exists="replace",
    index=False
)

print("Data loaded successfully into MySQL.")
```

Verify:

```
SELECT COUNT(*) AS total_rows
FROM tourism_data;
```


MySQL – Analysis Queries

1. **Total records**

```
SELECT COUNT(*) AS total_records
FROM tourism_data;
```

2. **Tourism performance by year**

```
SELECT
    year,
    SUM(tourism_arrivals) AS total_arrivals,
```

```
    SUM(tourism_receipts) AS total_receipts,
    SUM(tourism_expenditures) AS total_expenditures
FROM tourism_data
GROUP BY year
ORDER BY year;
```

year	total_arrivals	total_receipts	total_expenditures
1999	10103597948.589233	3278343598287.2666	1746.7010305810263
2000	10714186978.438171	3328750998384.577	1738.941154265043
2001	10418113117.98209	3230010034564.6167	1781.6699878636714
2002	10589341316.128107	3392392552250.0327	1847.8000356272262
2003	10493274452.547146	3814078912901.1523	1834.5318896734275
2004	12413507235.139769	4451164097769.923	1760.405749669151
2005	13002635038.327805	4848688070097.771	1710.2033802404376
2006	14194709839.179125	5209770870606.877	1684.4515188106088
2007	14753207229.550478	5992944285299.085	1669.3870213845976
2008	14912862070.064268	6571980039970.429	1603.8210260690817
2009	14420992093.431221	5973423336758.586	1702.5352764391594
2010	15015955067.17118	6471771638113.582	1630.4110287727488
2011	15295477562.88915	7193909229797.129	1557.325325338564
2012	15974784211.119461	7498735005532.441	1560.0994814058006
2013	16584436318.942139	8035324436204.887	1572.4452474391007
2014	16999350334.113457	8475189484605.114	1653.7213588919276
2015	17562723601.1548	8202026478519.081	1798.6622153088936
2016	18127828300.594513	8316999963337.126	1832.0984200715106
2017	19134543272.87612	9032419439346.121	1837.6773293182234
2018	19995522475.163628	9822005828758.541	1784.077843159268
2019	20603192950.096928	9960145931586.457	1829.5109826214584
2020	954576299.8532057	697460803612.2322	1287.6973932322398
2021	667128000	413098000000	1530.7740349626865
2022	667128000	413098000000	1530.7740349626865
2023	667128000	413098000000	1530.7740349626865

3. Top countries by tourism receipts

```
SELECT
    country,
    SUM(tourism_receipts) AS total_receipts,
    SUM(tourism_arrivals) AS total_arrivals
FROM tourism_data
GROUP BY country
ORDER BY total_receipts DESC
LIMIT 10;
```

	country	total_receipts	total_arrivals
▶	World	24073305702196.676	36911950266.92547
	High income	15404177736766.844	24161229680.496967
	Post-demographic dividend	14065108469185.621	20422591118.74985
	OECD members	13806284382106.043	22204358965.057682
	Europe & Central Asia	8733031847388.999	18979144618.60779
	European Union	6582407188041.448	15653516368.023464
	Euro area	5502834558763.997	8597131288.639877
	North America	4413148942179.611	3606546265.625
	United States	3526010000000	2937987265.625
	Early-demographic dividend	2852791375915.5474	5338037451.67985

4. **Tourism receipts vs expenditures**

```
SELECT
   year,
   ROUND(SUM(tourism_receipts), 2) AS total_receipts,
   ROUND(SUM(tourism_expenditures), 2) AS total_expenditures,
   ROUND(
      SUM(tourism_receipts) -
      SUM(tourism_expenditures), 2
   ) AS tourism_balance
FROM tourism_data
GROUP BY year
ORDER BY year;
```

	year	total_receipts	total_expenditures	tourism_balance
▶	1999	3278343598287.27	1746.7	3278343596540.57
	2000	3328750998384.58	1738.94	3328750996645.64
	2001	3230010034564.62	1781.67	3230010032782.95
	2002	3392392552250.03	1847.8	3392392550402.23
	2003	3814078912901.15	1834.53	3814078911066.62
	2004	4451164097769.92	1760.41	4451164096009.52
	2005	4848688070097.77	1710.2	4848688068387.57
	2006	5209770870606.88	1684.45	5209770868922.43
	2007	5992944285299.08	1669.39	5992944283629.7
	2008	6571980039970.43	1603.82	6571980038366.61
	2009	5973423336758.59	1702.54	5973423335056.05
	2010	6471771638113.58	1630.41	6471771636483.17
	2011	7193909229797.13	1557.33	7193909228239.8
	2012	7498735005532.44	1560.1	7498735003972.34
	2013	8035324436204.89	1572.45	8035324434632.44
	2014	8475189484605.11	1653.72	8475189482951.39
	2015	8202026478519.08	1798.66	8202026476720.42

2016	8316999963337.13	1832.1	8316999961505.03
2017	9032419439346.12	1837.68	9032419437508.44
2018	9822005828758.54	1784.08	9822005826974.46
2019	9960145931586.46	1829.51	9960145929756.94
2020	697460803612.23	1287.7	697460802324.53
2021	413098000000	1530.77	413097998469.23
2022	413098000000	1530.77	413097998469.23
2023	413098000000	1530.77	413097998469.23

5. **Average economic indicators**

```
SELECT
   year,
   ROUND(AVG(gdp), 2) AS average_gdp,
   ROUND(AVG(inflation), 2) AS average_inflation,
   ROUND(AVG(unemployment), 2) AS average_unemployment
FROM tourism_data
GROUP BY year
ORDER BY year;
```

year	average_gdp	average_inflation	average_unemployment
1999	906387913408.55	9.26	7.66
2000	936288549726.32	9.53	7.39
2001	933127359527.32	7.59	7.71
2002	969507194363.38	5.27	7.95
2003	1093511925440.13	5.22	7.69
2004	1241474139797.6	4.76	7.7
2005	1358227616732.09	5.4	7.58
2006	1489646727264	5.32	7.21
2007	1704724093236.7	5.37	7.14
2008	1901195378716.34	9.29	7.13
2009	1807544511839.38	3.86	7.74
2010	2005120387176.9	4.09	7.57
2011	2252138365003.89	6.07	7.41
2012	2309967087359.5	5.33	7.61
2013	2393778892501.17	4.16	7.67
2014	2465990841883.17	3.64	7.45
2015	2320327969031.98	3.71	7.46
2016	2352336326402.55	5.89	7.24
2017	2520263338447.36	4.64	7.26
2018	2683227644778.6	3.98	6.85
2019	2724030611453.07	4.59	6.78
2020	2644945045815.68	6.22	7.19
2021	3036001360197.41	6.85	7.11
2022	3160320471295.93	10.17	6.33
2023	3295470335981.86	7.84	6.31

3. Power BI – DAX Measures

Total Countries

Total Countries = DISTINCTCOUNT(tourism_data[country])

Total Tourist Arrivals

Total Tourist Arrivals =SUM(tourism_data[tourism_arrivals])

Total Tourism Receipts

Total Tourism Receipts = SUM(tourism_data[tourism_receipts])

Total Tourism Expenditure

Total Tourism Expenditure = SUM(tourism_data[tourism_expenditures])

Average GDP

Average GDP = AVERAGE(tourism_data[gdp])

Average Inflation

Average Inflation = AVERAGE(tourism_data[inflation])

Tourism Balance

Tourism Balance = [Total Tourism Receipts] - [Total Tourism Expenditure]

Power BI visuals created include KPI cards, tourist arrivals trend, tourism receipts versus expenditure, GDP-related analysis, and inflation versus unemployment.

Tableau Dashboard

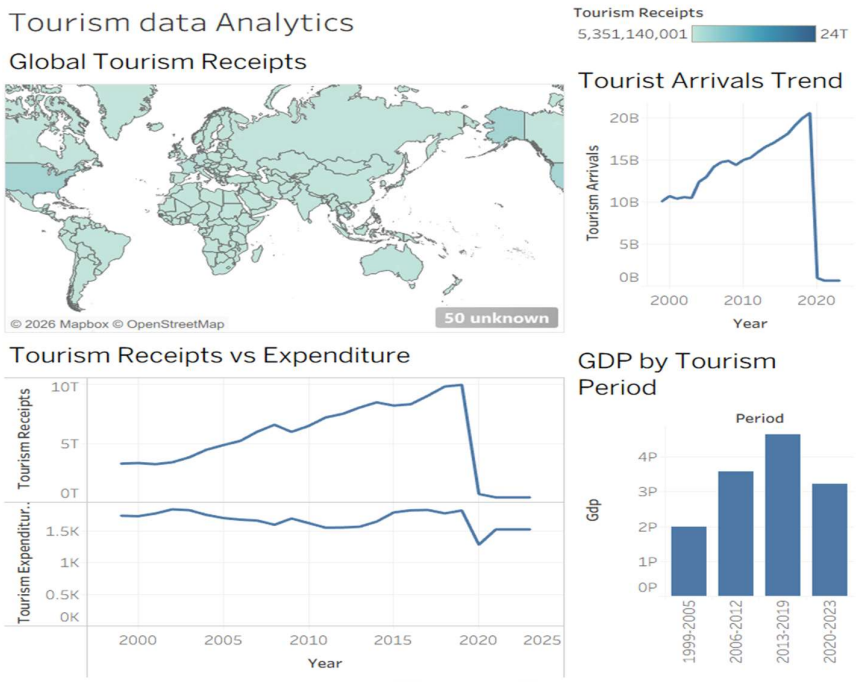
The cleaned CSV file was connected to Tableau Public and separate worksheets were created for tourism analysis. A world map was also created using Country with Tourism Receipts to show global tourism distribution.

Tableau worksheets used:

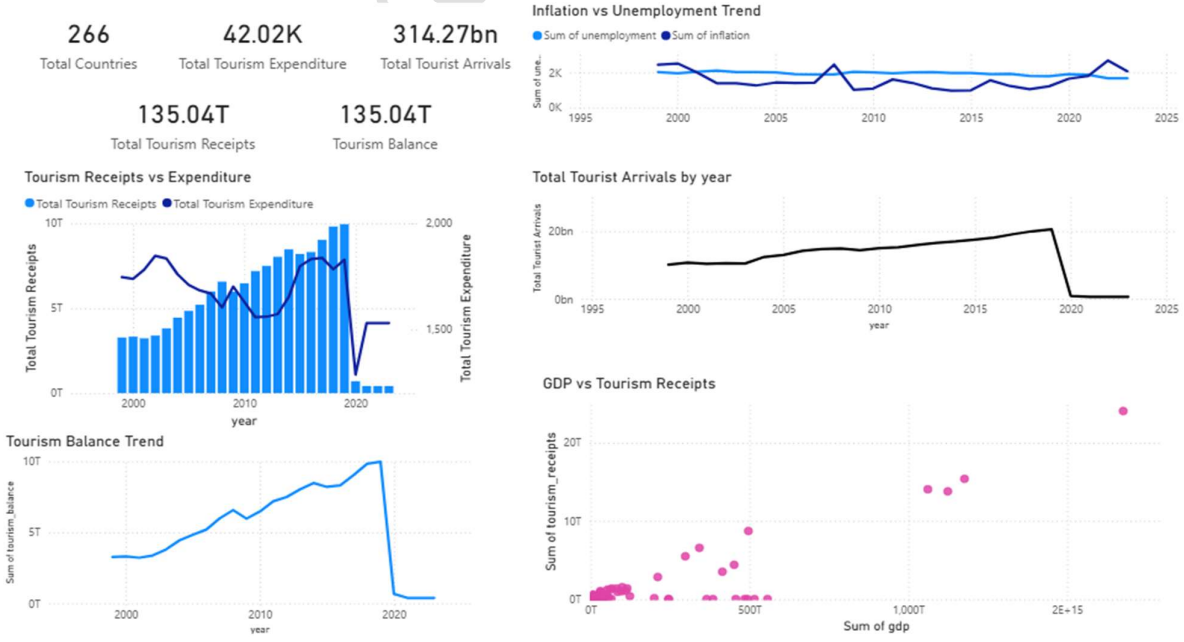
1. Tourist Arrivals Trend
2. Tourism Receipts vs Expenditure
3. Inflation vs Unemployment
4. Tourism Balance by Period
5. Global Tourism Receipts – World Map

Output:

TABLEAU DASHBOARD



POWER BI DASHBOARD



Result: